



服务管理和系统启动流程

誉天教育

- systemd系统管理
- Linux系统启动流程
- 重置root用户密码
- 修复系统故障（引导程序、引导配置文件、文件系统）

- systemd守护进程管理Linux启动，一般包括服务启动和服务管理。它可在系统引导时以及运行中的系统激活系统资源、服务器守护进程和其他进程。
- 守护进程是在执行各种任务的后台等待或运行的进程。一般情况下在系统引导时自动启动并持续运行至关机或手动停止。按照惯例，许多守护进程的名称以字母d结尾。
- 在红帽企业Linux中，第一个启动的进程（PID 1）是systemd，如下是systemd提供的项功能：
 - 1、并行化功能（同时启动多个服务），可提高系统的启动速度
 - 2、按需启动激活进程
 - 3、自动服务依赖管理

Init和systemd对比

- Init和systemd是Linux操作系统中不同的两种初始化系统
- Init是传统的初始化系统，它是一个独立的程序，负责启动操作系统的其他部分，init通过读取配hi文件确定启动顺序，并依次启动所需要的进程
- Systemd是一个综合性的工具，包括系统启动和服务管理

init	systemd
串行启动，前一个进程启动完成才启动下一个进程	并行启动，减少启动等待时间
内核加载，系统第一个进程（PID 1），位于/sbin/init	内核加载，系统第一个进程（PID 1），位于/usr/lib/systemd
Init进程执行启动脚本，脚本复杂	按需启动服务，简洁



- systemd使用单元来管理不同类型的对象
- 用systemctl -t type显示可用的单元类型，常用的如下：

类型	文件扩展名	解释
Service unit	.service	定义系统服务
Target unit	.target	模拟运行界别
Device unit	.device	定义内核识别的设备
Mount unit	.mount	定义文件系统挂载点
Socket unit	.socket	用于标识进程间通信
Swap unit	.swap	用于管理swap设备
Automount unit	.automount	用于实现文件系统自动挂载
Path unit	.path	定义文件系统的文件和目录

- 列出服务单元，用--type控制单元类型输出，--all列出所有单元，包括未激活的。

```
[user@rhel9 ~]$ systemctl list-units --type=service
```

UNIT	LOAD	ACTIVE	SUB	DESCRIPTION
accounts-daemon.service	loaded	active	running	Accounts Service
alsa-state.service	loaded	active	running	Manage Sound Card State (r
atd.service	loaded	active	running	Deferred execution schedul
auditd.service	loaded	active	running	Security Auditing Service
avahi-daemon.service	loaded	active	running	Avahi mDNS/DNS-SD Stack
chronyd.service	loaded	active	running	NTP client/server
colord.service	loaded	active	running	Manage, Install and Genera
crond.service	loaded	active	running	Command Scheduler
cups.service	loaded	active	running	CUPS Scheduler
dbus-broker.service	loaded	active	running	D-Bus System Message Bus

UNIT：服务单元的名称

LOAD：systemd是否正确解析了单元的配置，并将该单元加载到内存中

ACTIVE：单元的高级别激活状态，此信息表明单元是否已成功启动

SUB：单元的低级别激活状态，信息视单元类型、状态以及单元的执行方式而异。

DESCRIPTION：单元的简单描述

- systemctl list-units命令显示systemd服务解析并加载到内存中的单元，不显示已安装但未启用的服务。要查看所有已安装单元文件状态，请用systemctl list-unit-files

```
[user@rhel9 ~]$ systemctl list-unit-files --type=service
```

UNIT FILE	STATE	VENDOR PRESET
accounts-daemon.service	enabled	enabled
alsa-restore.service	static	-
alsa-state.service	static	-
arp-ethers.service	disabled	disabled
atd.service	enabled	enabled
auditd.service	enabled	enabled
auth-rpcgss-module.service	static	-
autovt@.service	alias	-
avahi-daemon.service	enabled	enabled

- STATE的类型
 - enabled：系统引导时启动
 - disabled：未设置为系统引导时启动
 - static：无法启动，但是可以由其他启动的单元引导启动
 - masked：无法启动，也无法被其他单元引导启动

启动/关闭/重启/重新加载服务	systemctl [start stop restart reload]	<name.service>
设置服务开机自启/关闭	systemctl [enable disable]	<name.service>
查看服务当前状态	systemctl status	<name.service>
查看服务启动状态	systemctl [is-enable is-active]	<name.service>

```
[root@rhel9 ~]# systemctl status sshd
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2023-10-10 09:13:39 CST; 1h 41min ago
```

```
[root@rhel9 ~]# systemctl is-enabled sshd
enabled
[root@rhel9 ~]# systemctl is-active sshd
active
```


- 重新加载服务

- ◆ `systemctl reload NAME.service` 重新加载配置文件而不重启服务
- ◆ `systemctl reload-or-restart NAME.service` 若不确定服务是否又重新加载配置文件的功能，可以用 `reload-or-restart`，如果支持重新加载则用 `reload`，反之，用 `restart`。

- 屏蔽服务设置服务开机时启动或不启动

- ◆ `systemctl mask NAME.service` 为防止系统服务之间冲突，可以屏蔽某个服务，防止管理员意外启动与其他服务冲突。
- ◆ `systemctl unmask NAME.service` 取消屏蔽服务
- ◆ 说明：禁用disabled的服务可以手动启动，或通过其他单元文件启动，但不会在系统开机时自动启动。屏蔽的服务无法手动启动，也不会自动启动。

- 列出服务的依赖性关系
 - ◆ systemctl list-dependencies sshd.service 列出启动服务时所需要的依赖性项的层次结构

```
[root@rhel9 ~]# systemctl list-dependencies sshd.service
sshd.service
├─system.slice
├─sshd-keygen.target
│   ├── sshd-keygen@ecdsa.service
│   ├── sshd-keygen@ed25519.service
│   └── sshd-keygen@rsa.service
├─sysinit.target
│   ├── dev-hugepages.mount
│   ├── dev-mqueue.mount
│   ├── dracut-shutdown.service
│   ├── iscsi-onboot.service
│   ├── kmod-static-nodes.service
│   ├── ldconfig.service
│   ├── lvm2-lvmpolld.socket
│   ├── lvm2-monitor.service
│   ├── multipathd.service
│   ├── nis-domainname.service
│   └── plymouth-read-write.service
```

- 重新启动和关闭系统

systemctl restart 重启系统，restart可以代替

systemctl poweroff关机，poweroff可以代替

- systemd目标是一组系统应启动达到所需状态的systemd单元，即系统运行级别，下表列出了重要的target。

target类型	描述
graphical.target	系统支持多用户、图形和基于文本的登录
multi-user.target	系统支持多用户、基于文本的登录
rescue.target	系统初始化已完成，需要root账户登录
emergency.target	只读挂载根文件系统，需要root账户登录

- 查看默认启动的运行目标：systemctl get-default
- 切换当前的运行目标：systemctl isolate NAME.target
 - ◆ 说明：切换到某个运行目标时，会停止该目标不需要的服务单元，并且启动该目标所需的服务单元。

```
[root@rhel9 ~]# systemctl isolate multi-user.target
```

- 设置下次开机的运行目标：systemctl set-default NAME.target

```
[root@rhel9 ~]# systemctl get-default  
graphical.target  
[root@rhel9 ~]# systemctl set-default multi-user.target  
Removed /etc/systemd/system/default.target.  
Created symlink /etc/systemd/system/default.target → /usr/lib/systemd/system/multi-user.target.
```

systemd目标 (target)

- 在系统启动时更改启动目标
- 具体步骤如下：
 - ◆ 重启系统
 - ◆ 将光标移动到要启动的内核
 - ◆ 按e编译当前条目
 - ◆ 将光标移至以linux开头的行，此为内核命令行
 - ◆ 添加systemd.unit=NAME.target
 - ◆ 按ctrl+x继续启动

```
load_video
set gfxpayload=keep
insmod gzio
linux ($root)/vmlinuz-5.14.0-70.13.1.el9_0.x86_64 root=/dev/mapper/rhel-root r\
o crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume=/dev/mapper/rhel-swap rd\
.lvm.lv=rhel/root rd.lvm.lv=rhel/swap rhgb quiet systemd.unit=graphical.target
initrd ($root)/initramfs-5.14.0-70.13.1.el9_0.x86_64.img
```


- 计算机接通电源，系统固件（BIOS）运行，开机自检（POST），并初始化部分硬件
- 系统固件按照顺序选择可启动的设备，搜索所有设备的主启动记录（MBR）
- 系统固件会从磁盘上读取boot loader，将系统控制权交给boot loader。红帽企业8系统中，boot loader为GRand Unified Bootloader Version 2（GRUB2）。
 - ◆ 使用 grub2-install命令可以在磁盘上安装GRUB2作为boot loader。
- GRUB2加载/boot/grub2/grub.cfg配置文件，选择要启动的内核
 - ◆ /boot/grub2/grub.cfg配置文件由/etc/grub.d/目录、/etc/default/grub文件和grub2-mkconfig命令配置
- boot loader加载内核和initramfs，并放入内存中，initramfs包含启动时硬件必要的内核模块和初始化脚本。

- boot loader将控制权交给内核，内核在initramfs中找到所有硬件的驱动程序，并初始化这些硬件，initramfs 中的 /sbin/init 以 PID 1 启动。在 RHEL8 中，/sbin/init 是一个指向 systemd 的链接。
- initramfs中的systemd会执行initrd.target目标所有的单元，包括将磁盘中的root文件系统挂载于/sysroot。
 - ◆ 使用/etc/fstab/配置
- 内核将根文件系统从initramfs切换到/sysroot中的根文件系统，随后systemd会使用磁盘中安装的systemd来重新执行。
- systemd最终会查找默认的启动目标，还会启动一个基于文本登录或者图形登录的界面。
 - ◆ 使用/etc/systemd/system/default.target和/etc/systemd/system/配置

修复/boot/grub2/grub.cfg

- 在RHEL中，/boot/grub2/grub.cfg文件丢失，系统无法正常启动：
- 修复步骤如下：

- ◆ 指定/boot目录所在的分区

```
grub> set root='hd0,msdos1'
```

- ◆ 加载内核文件，并以只读的方式加载根分区

```
grub> linux /vmlinuz-5.14.0-70.13.1.el9_0.x86_64 ro root=/dev/mapper/rhel-root
```

- ◆ 指定initramfs文件

```
grub> initrd /initramfs-5.14.0-70.13.1.el9_0.x86_64.img
```

- ◆ 启动

```
grub> boot_
```

- ◆ 进入系统后重新生成/boot/grub2/grub.cfg文件，重启测试

```
[root@rhel9 ~]# grub2-mkconfig -o /boot/grub2/grub.cfg
Generating grub configuration file ...
Adding boot menu entry for UEFI Firmware Settings ...
done
```

- ◆ 说明：每个系统的环境不同，设置方法也会不同。

- 查看默认启动内核

```
[root@rhel9 ~]# grubby --default-kernel  
/boot/vmlinuz-5.14.0-70.13.1.el9_0.x86_64
```

- 查看内核启动信息

- ◆ --info=ALL查看所有的内核启动项

```
[root@rhel9 ~]# grubby --info=/boot/vmlinuz-5.14.0-70.13.1.el9_0.x86_64  
index=0  
kernel="/boot/vmlinuz-5.14.0-70.13.1.el9_0.x86_64"  
args="ro crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume=/dev/mapper/rhel-swap rd.  
lvm.lv=rhel/root rd.lvm.lv=rhel/swap rhgb quiet"  
root="/dev/mapper/rhel-root"  
initrd="/boot/initramfs-5.14.0-70.13.1.el9_0.x86_64.img"  
title="Red Hat Enterprise Linux (5.14.0-70.13.1.el9_0.x86_64) 9.0 (Plow)"  
id="541e27f0837045fba5a1f78b45a5ad1c-5.14.0-70.13.1.el9_0.x86_64"
```

- ◆ 启动信息配置在/boot/loader/entries/目录下

- 修改默认启动内核：

- ◆ 方法一：根据菜单栏的索引号设置，从0开始

```
[root@rhel9 ~]# grubby --default-index  
0  
[root@rhel9 ~]# grub2-set-default 1  
[root@rhel9 ~]# grubby --default-index  
1
```

- ◆ 方法二：绑定特定版本内核

```
[root@rhel9 ~]# grubby --set-default /boot/vmlinuz-0-rescue-541e27f0837045fba5a1f78b45a5ad1c  
The default is /boot/loader/entries/541e27f0837045fba5a1f78b45a5ad1c-0-rescue.conf with index 1 and kernel /boot/vmlinuz-0-rescue-541e27f0837045fba5a1f78b45a5ad1c  
[root@rhel9 ~]# grubby --default-kernel  
/boot/vmlinuz-0-rescue-541e27f0837045fba5a1f78b45a5ad1c
```

- 内核参数定义在/etc/default/grub

- 用vim编辑时，需要更新grub2配置文件grub2-mkconfig -o /boot/grub2/grub.cfg

```
[root@rhel9 ~]# cat /etc/default/grub
GRUB_TIMEOUT=5
GRUB_DISTRIBUTOR="$(sed 's, release .*$,g' /etc/system-release)"
GRUB_DEFAULT=saved
GRUB_DISABLE_SUBMENU=true
GRUB_TERMINAL_OUTPUT="console"
GRUB_CMDLINE_LINUX="crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume=/dev/mapper/rhel-swap rd.
lvm.lv=rhel/root rd.lvm.lv=rhel/swap rhgb quiet"
GRUB_DISABLE_RECOVERY="true"
GRUB_ENABLE_BLSCFG=true
```

- 命令行修改内核参数

- 添加和修改参数用--args=console=ttyS0,115200，删除参数用--remove-args="rhgb quiet"

```
[root@rhel9 ~]# grubby --remove-args="rhgb quit" --args="console=ttyS0,115200" --update-kernel /boot/vmlinuz-5.14.0-70.13.1.el9_0.x86_64
```


- 在RHEL中，可以使从initramfs运行的脚本在某点暂停，并提供shell，然后在该shell下继续执行，这主要是为了进行调试，也可以使用该方法来重置丢失的root密码。
- 进入shell步骤：
 - ◆ 重启系统
 - ◆ 将光标移动到要启动的内核
 - ◆ 按e编译当前条目
 - ◆ 将光标移至以linux开头的行，此为内核命令行
 - ◆ 在末尾添加rd.break
 - ◆ 按ctrl+x继续启动

```
load_video
set gfxpayload=keep
insmod gzio
linux ($root)/vmlinuz-0-rescue-541e27f0837045fba5a1f78b45a5ad1c root=/dev/mapp\
er/rhel-root ro crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume=/dev/mappe\
r/rhel-swap rd.lvm.lv=rhel/root rd.lvm.lv=rhel/swap rhgb quiet rd.break_
initrd ($root)/initramfs-0-rescue-541e27f0837045fba5a1f78b45a5ad1c.img
```


- 重置root密码步骤：

- ◆ 重启以读写方式挂载/sysroot

```
switch_root:/# mount -o remount,rw /sysroot
```

- ◆ 切换至真正操作系统的根/sysroot

```
switch_root:/# chroot /sysroot
```

- ◆ 重置密码

```
sh-5.1# echo rehdat | passwd --stdin root
```

- ◆ 打标签

```
sh-5.1# touch /.autorelabel
```

- ◆ exit退出两次

```
sh-5.1# mount -o remount,rw /sysroot/  
sh-5.1# chroot /sysroot/  
sh-5.1# echo rehdat | passwd --stdin root  
Changing password for user root.  
passwd: all authentication tokens updated successfully.  
sh-5.1# touch /.autorelabel  
sh-5.1# exit  
exit  
sh-5.1# exit  
exit
```

- /etc/fstab中的错误和损坏的文件系统可能会阻止系统的启动，系统会进入需要提供root密码的紧急修复模式。
- 以下情况是常见的文件系统问题：
 - ◆ 文件系统损坏
 - ◆ /etc/fstab中引用的设备不存在
 - ◆ /etc/fstab中的文件系统类型书写错误

```
/dev/mapper/rhel-root    /                xfs      defaults    0 0
UUID=74df93c4-ef11-470d-8ca2-dee76cd88bf7 /boot           xfs      defaults    0 0
/dev/mapper/rhel-home    /home           xfs      defaults    0 0
/dev/mapper/rhel-swap    none            swap     defaults    0 0
UUID="eccfaa21-497d-4fa6-aff9-d5d662a9be45" /data           ext4     defaults    0 0
~
```

- 修复步骤如下：

- 进入紧急模式，输入root密码

```
You are in emergency mode. After logging in, type "journalctl -xb" to view
system logs, "systemctl reboot" to reboot, "systemctl default" or "exit"
to boot into default mode.
Contraseña de root para mantenimiento      输入root密码
(o pulse Control-D para continuar):
[root@rhel9 ~]#
```

- 尝试挂载所有的文件系统，定位/etc/fstab中的问题。

```
[root@rhel9 ~]# mount -a
[ 70.673630] EXT4-fs (sda1): VFS: Can't find ext4 filesystem
mount: /data: tipo de sistema de ficheros incorrecto, opción incorrecta, superbloque incorrecto en /
dev/sda1, falta la página de códigos o el programa auxiliar, o algún otro error.
```

- 确保当前根文件系统可读写，若是ro，需要改成rw

```
[root@rhel9 ~]# mount | grep -w /
/dev/mapper/rhel-root on / type xfs (rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,noquota)
```

- 修改成正确的配置

```
[root@rhel9 ~]# lsblk -f
NAME FSTYPE FSVER LABEL UUID                                FSAVAIL FSUSE% MOUNTPOINTS
sda
└─sda1
   xfs                                eccfaa21-497d-4fa6-aff9-d5d662a9be45
```

```
[root@rhel9 ~]# grep ecc /etc/fstab
UUID="eccfaa21-497d-4fa6-aff9-d5d662a9be45" /data xfs defaults 0 0
```

Boot loader损坏修复

- 修复步骤如下：

- ◆ 模拟破坏引导程序

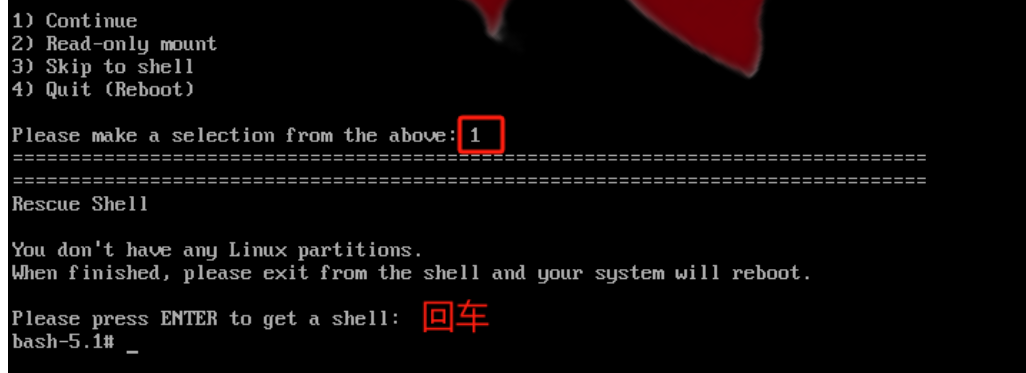
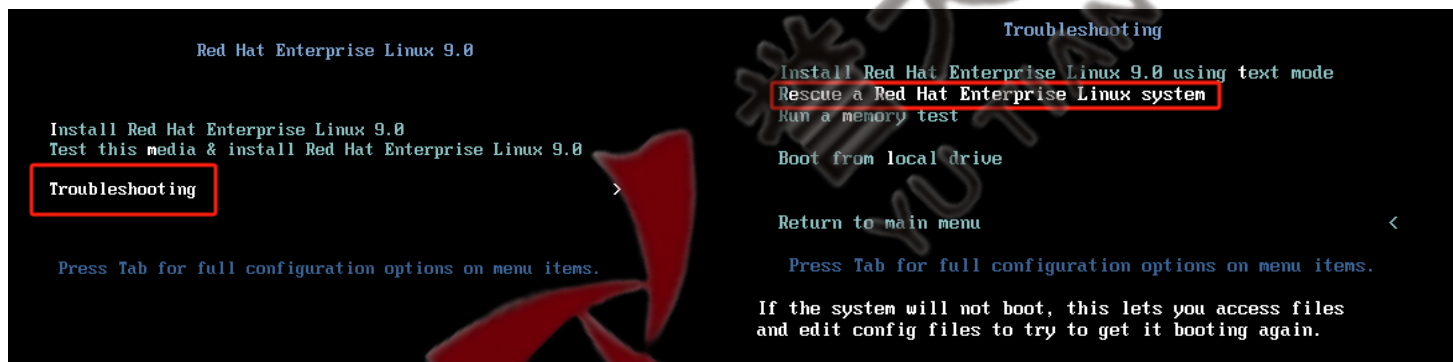
```
[root@rhel9 ~]# dd if=/dev/zero of=/dev/nvme0n1 bs=446 count=1
```

记录了1+0 的读入

记录了1+0 的写出

446字节已复制，0.000607473 s，734 kB/s

- ◆ 选择从光盘启动进入救援模式



- 修复步骤如下：
 - ◆ 切换至真正的根文件系统

```
bash-5.1# chroot /mnt/sysimage/  
bash-5.1#
```

- ◆ 修复引导程序

```
bash-5.1# grub2-install /dev/nvme0n1  
Installing for i386-pc platform.  
Installation finished. No error reported.
```

- ◆ exit退出2次，重启测试，选择从本地磁盘启动

- systemd系统管理
- Linux系统启动流程
- 重置root用户密码
- 修复系统故障（引导程序、引导配置文件、文件系统）

